



Cofinancé par
l'Union européenne



Soutenu par
Invest for Jobs
Opportunités de croissance en Afrique

Mise en œuvre par
giz Institut International
de Coopération (IIC) GmbH



RAPPORT DE PROJET

Développement d'un système intelligent de vérification automatique de faits basé sur l'IA (Fact-Checker AI)

Nom : ASSAN CYRIAC ABONOUAN

Établissement : TECH TALENT ACCELERATOR

Formation : GenAI & Machine Learning

Formateur : Mr. ELINGUI PASCAL URIEL

Date : 29/07/2026

Résumé

Ce projet vise à concevoir et développer un système de vérification automatique de faits (Fact-Checker AI), capable d'évaluer la véracité d'une affirmation en s'appuyant sur un corpus documentaire Wikipedia indexé. Le système repose sur une architecture de type recherche augmentée (RAG) associant recherche sémantique par embeddings (Sentence Transformers, modèle all-MiniLM-L6-v2), un index vectoriel FAISS, un module de reranking par cross-encoder (ms-marco-MiniLM-L-6-v2) et un module de vérification par inférence de langage naturel (NLI, modèle nli-deberta-v3-base).

Le pipeline complet, orchestré par la classe FactCheckerPipeline, transforme une affirmation en un verdict (SUPPORTS, REFUTES ou NOT ENOUGH INFO) accompagné des preuves retrouvées et d'un score de confiance, restitué à l'utilisateur via une interface Streamlit.

Le système a été évalué sur un échantillon du jeu de données FEVER : sur 50 affirmations testées, il atteint une exactitude (accuracy) de 64 %, un taux de rappel des preuves (Evidence Recall) de 56 % et un FEVER Score de 38 %. Ce rapport présente le contexte du projet, la problématique traitée, l'architecture et le fonctionnement détaillé du système, son interface utilisateur, les résultats d'évaluation obtenus, ainsi que les difficultés rencontrées, les limites actuelles et les perspectives d'évolution.

1. Contexte

- L'information circule aujourd'hui à une vitesse et dans un volume sans précédent, notamment via Internet et les réseaux sociaux.
- Cette expansion rapide s'accompagne d'une propagation tout aussi rapide de fausses informations (« fake news »), parfois difficiles à distinguer de sources fiables.
- Face à ce flux constant, les utilisateurs disposent rarement du temps ou des outils nécessaires pour vérifier rapidement et de manière fiable la véracité d'une affirmation.
- Ce contexte a conduit au développement de systèmes automatisés de fact-checking, capables de croiser une affirmation avec une base documentaire de référence afin d'en évaluer la véracité de manière rapide, reproductible et transparente.

2. Problématique

Comment développer un système capable de vérifier automatiquement la véracité d'une affirmation en s'appuyant sur une base documentaire fiable, tout en fournissant des preuves et une explication compréhensible?

3. Solution proposée

Fact-Checker AI est un système de vérification automatique de faits qui s'appuie sur un corpus documentaire Wikipedia indexé, et sur le jeu de données FEVER (Fact Extraction and VERification) pour son évaluation.

Principe du système

À partir d'une affirmation (claim) soumise par l'utilisateur, le système recherche dans le corpus indexé les passages les plus pertinents, les trie par pertinence, puis analyse chaque passage retenu à l'aide d'un modèle d'inférence de langage naturel (NLI) afin de déterminer s'il confirme, contredit, ou ne permet pas de statuer sur l'affirmation. Les résultats individuels sont ensuite agrégés pour produire un verdict global assorti d'un score de confiance.

Fonctionnement global

Affirmation → Recherche (retrieval) → Tri des preuves (reranking) → Vérification (NLI) → Verdict et preuves.

Utilisateurs ciblés

Toute personne souhaitant vérifier rapidement une affirmation à l'aide de sources documentaires (contexte pédagogique, veille informationnelle, aide à la vérification). Le système est explicitement présenté, dans son interface, comme une aide à la vérification et non comme une source de vérité unique.

Bénéfices

- Transparence : chaque verdict est accompagné des preuves et du document source utilisés.

- Traçabilité : chaque étape du pipeline (recherche, tri, analyse) est indépendante et peut être vérifiée séparément.
- Accessibilité : interface web simple ne nécessitant aucune compétence technique.

4. Objectifs

Objectif général

Concevoir et développer un pipeline complet de vérification automatique de faits, combinant recherche d'information sémantique et vérification par intelligence artificielle, restitué via une interface utilisateur accessible.

Objectifs spécifiques

- Prétraiter le corpus Wikipedia (segmentation en phrases et en chunks) — indexing/preprocess_wikipedia.py
- Construire une base documentaire structurée — indexing/build_chunk_database.py (base SQLite chunks.db)
- Générer les représentations vectorielles (embeddings) des passages — indexing/build_embeddings.py
- Construire un index de recherche vectorielle FAISS — indexing/build_faiss.py
- Retrouver les documents et passages pertinents pour une affirmation donnée — retrieval/document_retriever.py, retrieval/passage_retriever.py
- Trier les preuves retrouvées par pertinence (reranking) — retrieval/reranker.py
- Vérifier une affirmation grâce à l'IA (inférence de langage naturel) — verification/verifier.py
- Générer une explication du verdict à travers les preuves et les scores de confiance associés — frontend/app.py

5. Technologies utilisées

Technologie	Utilisation
Python	Langage de développement principal du projet
Streamlit	Interface utilisateur web
FAISS (faiss-cpu)	Recherche vectorielle (index de similarité)
Sentence Transformers	Génération des embeddings (modèle all-MiniLM-L6-v2)
Cross-Encoder	Reranking des preuves (modèle ms-marco-MiniLM-L-6-v2)
Transformers (Hugging Face)	Vérification NLI (modèle nli-deberta-v3-base)
PyTorch	Backend d'exécution des modèles de deep learning
NumPy	Manipulation et stockage des embeddings
SQLite	Base de données des passages indexés (chunks.db)
FEVER	Jeu de données d'évaluation (affirmations, preuves, labels)

6. Architecture générale

```
1
2 @dataclass(frozen=True)
3 class PipelineSettings:
4     retrieval_top_k: int = DEFAULT_RETRIEVAL_TOP_K
5     rerank_top_k: int = DEFAULT_RERANK_TOP_K
6     max_chunks_per_document: int = DEFAULT_MAX_CHUNKS_PER_DOCUMENT
7
8
9 class FactCheckerPipeline:
10     def __init__(
11         self,
12         document_retriever: DocumentRetriever | None = None,
13         passage_retriever: PassageRetriever | None = None,
14         reranker: Reranker | None = None,
15         verifier: Verifier | None = None,
16         settings: PipelineSettings | None = None,
17     ):
18         self.settings = settings or PipelineSettings()
19         self.document_retriever = document_retriever or DocumentRetriever()
20         self.passage_retriever = passage_retriever or PassageRetriever(
21             self.document_retriever
22         )
23         self.reranker = reranker or Reranker()
24         self.verifier = verifier or Verifier()
25
26     def verify_claim(self, claim: str) -> dict:
27         claim = claim.strip()
28         if not claim:
29             raise ValueError("Claim must not be empty.")
30
31         passages = self.passage_retriever.retrieve(
32             query=claim,
33             top_k=self.settings.retrieval_top_k,
34             max_chunks_per_document=self.settings.max_chunks_per_document,
35         )
36         reranked_passages = self.reranker.rerank(
37             query=claim,
38             passages=passages,
39             top_k=self.settings.rerank_top_k,
40         )
41         verification = self.verifier.verify(
42             claim=claim,
43             evidence=reranked_passages,
44         )
45
46         return {
47             "claim": claim,
48             "label": verification["label"],
49             "confidence": verification["confidence"],
50             "label_scores": verification["label_scores"],
51             "best_evidence": verification["best_evidence"],
52             "evidence": verification["evidence"],
53             "retrieved_passages": passages,
54             "reranked_passages": reranked_passages,
55             "settings": {
56                 "retrieval_top_k": self.settings.retrieval_top_k,
57                 "rerank_top_k": self.settings.rerank_top_k,
58                 "max_chunks_per_document": self.settings.max_chunks_per_document,
59             },
60         }
61
62     def close(self) -> None:
63         self.document_retriever.close()
64
```

Le pipeline de vérification est orchestré par la classe FactCheckerPipeline (api/pipeline.py), qui enchaîne quatre composants indépendants: DocumentRetriever (recherche vectorielle), PassageRetriever (mise en forme et diversification des passages), Reranker (tri fin par cross-

7.2 Génération des embeddings

```
1
2 def load_chunks(input_file: Path) -> Iterator[dict]:
3     """
4     Lit les chunks un par un depuis le fichier JSONL.
5     """
6
7     with input_file.open(
8         "r",
9         encoding="utf-8"
10    ) as f:
11        for line in f:
12            if line.strip():
13                yield json.loads(line)
14
15 def load_model(model_name: str) -> SentenceTransformer:
16     """
17     Charge le modèle SentenceTransformer utilisé
18     pour générer les embeddings.
19     """
20     print("Chargement du modèle...")
21
22     model = SentenceTransformer(model_name)
23
24     print("Modèle chargé.")
25
26     return model
27
28
29 def encode_batch(
30     model: SentenceTransformer,
31     texts: list[str]
32 ) -> np.ndarray:
33     """
34     Encode un batch de textes en embeddings.
35     """
36
37     embeddings = model.encode(
38         texts,
39         batch_size=len(texts),
40         show_progress_bar=False, # la pipeline aura déjà sa propre progression dans main().
41         convert_to_numpy=True, # on évite donc une conversion supplémentaire,
42         normalize_embeddings=True # En normalisant des cette étape : on évite de le refaire plus tard ; la recherche est plus rapide ; les scores sont plus cohérents. C'est une pratique courante avec les modèles Sentence Transformers.
43     )
44
45     return embeddings
46
47
48
```

Le script `indexing/build_embeddings.py` charge le modèle `SentenceTransformer all-MiniLM-L6-v2` et encode les chunks par lots de 512 (`BATCH_SIZE = 512`). Les embeddings sont normalisés au moment de l'encodage, ce qui permet ensuite une recherche par similarité cosinus efficace. Les résultats sont sauvegardés sous forme de plusieurs fichiers `.numpy`, accompagnés d'un fichier de métadonnées (`metadata.json`) décrivant le modèle utilisé et la dimension des vecteurs.

7.3 Construction de l'index FAISS

```
1
2 def load_metadata(metadata_file: Path) -> dict:
3     """
4     Charge les métadonnées des embeddings.
5     """
6
7     with metadata_file.open("r",encoding="utf-8") as f:
8
9         return json.load(f)
10
11 def load_embedding_files(embeddings_dir: Path) -> Iterator[Path]:
12     """
13     Parcourt tous les fichiers d'embeddings (.npz)
14     dans l'ordre de leur création.
15     """
16
17     for file_path in sorted(embeddings_dir.glob("embeddings_*.npz")):
18         yield file_path
19
20
21 def create_index(embedding_dimension: int) -> faiss.Index:
22     """
23     Crée un index FAISS basé sur
24     la similarité cosinus.
25     """
26
27     return faiss.IndexFlatIP(embedding_dimension) #tous les vecteurs ont une norme égale à 1
28
29 def add_embeddings(index: faiss.Index, embedding_file: Path) -> int:
30     """
31     Charge un fichier d'embeddings
32     et les ajoute à l'index FAISS.
33
34     Returns
35     -----
36     int
37         Nombre de vecteurs ajoutés.
38     """
39
40     embeddings = np.load(
41         embedding_file
42     ).astype(np.float32)
43
44     index.add(
45         embeddings
46     )
47
48     return embeddings.shape[0]
49
50 def save_index(index: faiss.Index,output_file: Path) -> None:
51     """
52     Sauvegarde l'index FAISS.
53     """
54
55     faiss.write_index(
56         index,
57         str(output_file)
58     )
59
60
```

Construction de l'index FAISS

Le script `indexing/build_faiss.py` charge l'ensemble des fichiers d'embeddings générés, construit un index FAISS de type `IndexFlatIP` (produit scalaire, équivalent à une similarité cosinus puisque les vecteurs sont normalisés), puis sauvegarde l'index dans `data/indexes/faiss.index`.

7.4 Recherche des documents


```

1
2 def _missing_dependency_message(package: str) -> str:
3     return f"Missing dependency '{package}'. Activate the project environment before running retrieval, for example: conda activate fact-checker."
4
5 def load_index(index_file: Path = FAISS_INDEX_FILE) -> Any:
6     if not index_file.exists():
7         raise FileNotFoundError(f"FAISS index not found: {index_file}")
8
9     try:
10         import faiss
11     except ImportError as exc:
12         raise ImportError(_missing_dependency_message("faiss")) from exc
13
14     return faiss.read_index(str(index_file))
15
16 def load_model(model_name: str = EMBEDDING_MODEL_NAME) -> Any:
17     try:
18         from sentence_transformers import SentenceTransformer
19     except ImportError as exc:
20         raise ImportError(_missing_dependency_message("sentence-transformers")) from exc
21
22     return SentenceTransformer(model_name)
23
24 def encode_query(model: Any, query: str) -> np.ndarray:
25     query = query.strip()
26
27     if not query:
28         raise ValueError("Query must not be empty.")
29
30     embedding = model.encode(
31         [query],
32         convert_to_numpy=True,
33         normalize_embeddings=True,
34         show_progress_bar=False,
35     )
36
37     return embedding.astype(np.float32)
38
39 def search_index(index: Any, query_embedding: np.ndarray, top_k: int) -> tuple[np.ndarray, np.ndarray]:
40
41     if top_k <= 0:
42         raise ValueError("top_k must be greater than zero.")
43
44     return index.search(query_embedding, top_k)
45
46 def load_database(database_file: Path = CHUNKS_DATABASE_FILE) -> sqlite3.Connection:
47     if not database_file.exists():
48         raise FileNotFoundError(f"SQLite chunks database not found: {database_file}")
49
50     connection = sqlite3.connect(str(database_file))
51     connection.row_factory = sqlite3.Row
52
53     return connection
54
55 def get_chunk(connection: sqlite3.Connection, chunk_id: int) -> dict | None:
56
57     cursor = connection.execute(
58         """
59         SELECT
60             chunk_id,
61             doc_id,
62             title,
63             source,
64             start_sentence,
65             end_sentence,
66             sentence_ids,
67             num_sentences,
68             text_length,
69             text
70         FROM chunks
71         WHERE chunk_id = ?
72         """,
73         (chunk_id,),
74     )
75
76     row = cursor.fetchone()
77
78     if row is None:
79         return None
80
81     return {
82         "chunk_id": row["chunk_id"],
83         "doc_id": row["doc_id"],
84         "title": row["title"],
85         "source": row["source"],
86         "start_sentence": row["start_sentence"],
87         "end_sentence": row["end_sentence"],
88         "sentence_ids": json.loads(row["sentence_ids"]),
89         "num_sentences": row["num_sentences"],
90         "text_length": row["text_length"],
91         "text": row["text"],
92     }
93
94

```

Recherche de documents

Lorsqu'une affirmation est soumise, DocumentRetriever (retrieval/document_retriever.py) encode la requête avec le même modèle d'embeddings que celui utilisé pour l'indexation, puis interroge l'index FAISS pour récupérer les passages les plus proches sémantiquement. Les chunks correspondants sont ensuite récupérés depuis la base SQLite (chunks.db). PassageRetriever (retrieval/passage_retriever.py) limite ensuite le nombre de passages provenant d'un même document (par défaut 3), afin de diversifier les sources retenues.

7.5 Reranking

```
1 from typing import Any
2
3 from config.setting import RERANKER_MODEL_NAME
4
5 class Reranker:
6     def __init__(self, model_name: str = RERANKER_MODEL_NAME, model: Any | None = None):
7         if model is not None:
8             self.model = model
9             self.model_name = model_name
10            return
11
12        try:
13            from sentence_transformers import CrossEncoder
14        except ImportError as exc:
15            raise ImportError(
16                "Missing dependency 'sentence-transformers'. Activate the project "
17                "environment before running reranking."
18            ) from exc
19
20        self.model_name = model_name
21        self.model = CrossEncoder(model_name)
22
23    def rerank(self, query: str, passages: list[dict], top_k: int = 5) -> list[dict]:
24        query = query.strip()
25        if not query:
26            raise ValueError("Query must not be empty.")
27        if top_k <= 0:
28            raise ValueError("top_k must be greater than zero.")
29        if not passages:
30            return []
31
32        pairs = [(query, passage["text"]) for passage in passages]
33        scores = self.model.predict(pairs)
34
35        reranked_results = []
36        for passage, score in zip(passages, scores):
37            result = passage.copy()
38            result["rerank_score"] = float(score)
39            reranked_results.append(result)
40
41        reranked_results.sort(key=lambda item: item["rerank_score"], reverse=True)
42        return reranked_results[:top_k]
43
```

Reranking des preuves

Le Reranker (retrieval/reranker.py) utilise un modèle Cross-Encoder (ms-marco-MiniLM-L-6-v2) qui évalue directement chaque paire (affirmation, passage), ce qui est plus précis qu'une simple similarité vectorielle. Seuls les meilleurs passages (par défaut 5) sont conservés pour l'étape de vérification.

7.6 Vérification (NLI)

```
1
2 class Verifier:
3     def __init__(
4         self,
5         model_name: str = NLI_MODEL_NAME,
6         tokenizer: Any | None = None,
7         model: Any | None = None,
8     ):
9         self.model_name = model_name
10
11         if tokenizer is not None and model is not None:
12             self.tokenizer = tokenizer
13             self.model = model
14         else:
15             from transformers import AutoModelForSequenceClassification, AutoTokenizer
16
17             self.tokenizer = AutoTokenizer.from_pretrained(model_name)
18             self.model = AutoModelForSequenceClassification.from_pretrained(model_name)
19
20         self.model.eval()
21         self.id2label = dict(self.model.config.id2label)
22
23     def predict(self, claim: str, evidence: str) -> dict:
24         return self.predict_batch(claim, [evidence])[0]
25
26     def predict_batch(self, claim: str, evidence_texts: list[str]) -> list[dict]:
27         claim = claim.strip()
28         if not claim:
29             raise ValueError("Claim must not be empty.")
30
31         evidence_texts = [text.strip() for text in evidence_texts]
32         if any(not text for text in evidence_texts):
33             raise ValueError("Evidence text must not be empty.")
34         if not evidence_texts:
35             return []
36
37         inputs = self.tokenizer(
38             evidence_texts,
39             [claim] * len(evidence_texts),
40             return_tensors="pt",
41             truncation=True,
42             max_length=512,
43             padding=True,
44         )
45
46         with torch.no_grad():
47             outputs = self.model(**inputs)
48
49         probabilities = torch.softmax(outputs.logits, dim=-1)
50         predicted_ids = torch.argmax(probabilities, dim=-1)
51
52         results = []
53         for row_probabilities, predicted_id_tensor in zip(probabilities, predicted_ids):
54             predicted_id = predicted_id_tensor.item()
55             results.append(
56                 {
57                     "label": self.id2label[predicted_id],
58                     "confidence": row_probabilities[predicted_id].item(),
59                     "probabilities": row_probabilities.tolist(),
60                 }
61             )
62         return results
63
64     def convert_to_fever_label(self, nli_label: str) -> str:
65         label = nli_label.lower()
66
67         if "entail" in label:
68             return "SUPPORTS"
69         if "contradiction" in label or "contradict" in label:
70             return "REFUTES"
71         return "NOT ENOUGH INFO"
72
73     def analyze_passage(self, claim: str, passage: dict) -> dict:
74         return self.analyze_passages(claim=claim, passages=[passage])[0]
75
```

Vérification NLI

Le Verifier (verification/verifier.py) utilise un modèle NLI (nli-deberta-v3-base) pour classer chaque paire (preuve, affirmation) en entailment (confirme), contradiction (contredit) ou neutral (ne permet pas de statuer). Ces labels sont convertis vers la nomenclature FEVER (SUPPORTS, REFUTES, NOT ENOUGH INFO). Les prédictions obtenues pour l'ensemble des preuves retenues sont ensuite agrégées (somme des scores de confiance par label) pour déterminer le verdict final et son niveau de confiance.

7.7 Génération de l'explication

```
1
2 def initialize_state() -> None:
3     st.session_state.setdefault("page", PAGE_CHECK)
4     st.session_state.setdefault("claim_text", "")
5     st.session_state.setdefault("current_result", None)
6     st.session_state.setdefault("history", [])
7
8
9 def set_page(page: str) -> None:
10     st.session_state.page = page
11
12
13 def set_claim(claim: str) -> None:
14     st.session_state.claim_text = claim
15
16
17 def random_claim() -> None:
18     st.session_state.claim_text = random.choice(EXAMPLE_CLAIMS)
19
20
21 def clear_claim() -> None:
22     st.session_state.claim_text = ""
23
24
25 def label_class(label: str) -> str:
26     if label == "SUPPORTS":
27         return "supports"
28     if label == "REFUTES":
29         return "refutes"
30     return "nei"
31
32
33 def label_display(label: str) -> str:
34     if label == "SUPPORTS":
35         return "CONFIRME"
36     if label == "REFUTES":
37         return "CONTREDIT"
38     return "PREUVES INSUFFISANTES"
39
40
41 def percent(value: float | None) -> str:
42     return f"({value or 0.0} * 100:.0f)%"
43
44
45 def label_progress_class(label: str) -> str:
46     return label_class(label)
47
48
49 def truncate_text(text: str, length: int = 130) -> str:
50     text = " ".join(str(text).split())
51     if len(text) <= length:
52         return text
53     return f"{text[: length - 1].rstrip()}..."
54
55
56 @st.cache_data(show_spinner=False)
57 def load_artifact_status() -> dict:
58     status = {
59         "faiss_index": FAISS_INDEX_FILE.exists(),
60         "sqlite_database": CHUNKS_DATABASE_FILE.exists(),
61         "processed_chunks": PROCESSED_CHUNKS_FILE.exists(),
62         "embeddings_metadata": EMBEDDINGS_METADATA_FILE.exists(),
63         "chunk_count": None,
64         "document_count": None,
65     }
66
67     if CHUNKS_DATABASE_FILE.exists():
68         connection = sqlite3.connect(CHUNKS_DATABASE_FILE)
69         try:
70             row = connection.execute(
71                 "SELECT COUNT(*), COUNT(DISTINCT doc_id) FROM chunks"
72             ).fetchone()
73             status["chunk_count"] = row[0]
74             status["document_count"] = row[1]
75         finally:
76             connection.close()
77
78     return status
79
80
81 @st.cache_resource(show_spinner="Chargement des modèles de recherche, de tri et d'analyse...")
82 def load_model_bundle() -> tuple[DocumentRetriever, PassageRetriever, Reranker, Verifier]:
83     document_retriever = DocumentRetriever()
84     passage_retriever = PassageRetriever(document_retriever)
85     reranker = Reranker()
86     verifier = Verifier()
87     return document_retriever, passage_retriever, reranker, verifier
88
89
90 def load_pipeline(
91     retrieval_top_k: int,
92     rerank_top_k: int,
93     max_chunks_per_document: int,
94 ) -> FactCheckerPipeline:
95     document_retriever, passage_retriever, reranker, verifier = load_model_bundle()
96     settings = PipelineSettings(
97         retrieval_top_k=retrieval_top_k,
98         rerank_top_k=rerank_top_k,
99         max_chunks_per_document=max_chunks_per_document,
100     )
101     return FactCheckerPipeline(
102         document_retriever=document_retriever,
103         passage_retriever=passage_retriever,
104         reranker=reranker,
105         verifier=verifier,
106         settings=settings,
107     )
108
```

Explication du verdict

L'interface Streamlit (frontend/app.py) restitue, pour chaque verdict, le document source, le passage utilisé et le niveau de confiance de l'analyse pour chaque preuve retenue. L'« explication » fournie par le système repose ainsi sur la transparence des preuves et des scores associés, plutôt que sur un texte explicatif généré par un modèle de langage génératif.

8. Interface utilisateur

L'interface, développée avec Streamlit (frontend/app.py), est organisée en plusieurs pages accessibles depuis un menu latéral : « Vérifier une affirmation », « Résultat », « Historique », « Comment ça marche » et « Fiabilité du système ». Un panneau de réglages avancés permet d'ajuster le nombre de sources explorées, le nombre de preuves retenues et le nombre maximal de passages par document source.

Page d'accueil / Vérifier une affirmation



Cette page présente le principe du système et permet de saisir une affirmation à vérifier.

Saisie d'une affirmation



Un champ de texte permet de saisir librement une affirmation, avec des exemples cliquables proposés par défaut. L'interface précise explicitement que le corpus et les modèles ne comprennent que l'anglais.

Résultat

Résultat de la vérification

AFFIRMATION

CONTREDIT

“The Earth is flat.”

58%

Score de confiance

IA

Preuve principale : A Flat, G Flat and C (CONTREDIT, confiance 100%).

Ce système est un prototype et peut se tromper (environ 62% d'exactitude mesurée) — vérifiez toujours auprès d'autres sources. Détails sur la page « Fiabilité du système » (menu de gauche).

Résultat de la vérification

Le verdict final (Confirme / Contredit / Preuves insuffisantes) est affiché avec son score de confiance sous forme de progression.

Preuves

Preuves utilisées pour ce verdict

P1

A Flat -LRB-film-RRB-

Confiance 100%

“A Flat is a Hindi thriller film , directed by Hemant Madhukar and produced by Anjum Rizvi.The film was released on 12 November 2010 under the Anjum Rizvi Film Company and Y.T Entertainment Ltd. banners . Hemant Madhukar Hemant Madhukar Hindi Hindi language thriller film thriller film”

PREUVES INSUFFISANTES

P2

Angles between flats

Confiance 98%

“If the flats do not intersect , their shortest distance is one more invariant . flats flat (geometry) invariant invariant (mathematics) distance distance These angles are called canonical or principal . The concept of angles can be generalised to pairs of flats in a finite-dimensional inner product space over the complex numbers . space space flats flat (geometry) inner product space inner product space”

PREUVES INSUFFISANTES

P3

Almost flat manifold

Confiance 97%

Preuves retrouvées

Chaque preuve retenue est présentée sous forme de carte indiquant le document source, le passage extrait et le score de confiance associé à son analyse individuelle.

Explication / Comment ça marche



Cette page décrit visuellement les six étapes du pipeline (Affirmation, Recherche, Tri, Preuves, Analyse, Verdict) ainsi que la taille de la base de connaissances disponible.

9. Tests et résultats

Exemples de vérification

Les exemples suivants ont été obtenus en exécutant le pipeline complet (FactCheckerPipeline.verify_claim) sur l'environnement de développement (CPU, sans GPU) :

Affirmation	Verdict	Confiance	Temps d'exécution
The Eiffel Tower is located in Paris.	SUPPORTS	80 %	79,96 s
Albert Einstein won the Nobel Prize in Physics.	SUPPORTS	60 %	32,66 s
The Earth is flat.	REFUTES	58 %	22,66 s

Temps moyen observé par affirmation : 45,09 secondes (hors chargement initial des modèles). Le chargement initial des trois modèles (embeddings, reranker, NLI) prend environ 66,73 secondes.

Évaluation sur le jeu de données FEVER

Le pipeline complet a été évalué sur un échantillon de 50 affirmations du jeu de développement FEVER (shared_task_dev.jsonl), à l'aide du script evaluation/evaluate_pipeline.py. Sur les 1 196 premiers exemples parcourus, 1 146 ont été ignorés (affirmations sans preuve de référence, ou dont la preuve de référence ne figure pas dans le corpus indexé), jusqu'à obtenir 50 affirmations évaluables.

Métrique	Valeur	Signification
Accuracy	64,00 %	Part des verdicts finaux corrects
Evidence Recall	56,00 %	Part des affirmations pour lesquelles une preuve correcte a été retrouvée
Evidence Precision	3,69 %	Part des passages retenus qui correspondent exactement à une preuve de référence
FEVER Score	38,00 %	Verdict ET preuve corrects simultanément (métrique officielle FEVER)
Macro F1	0,4423	Moyenne des F1 par classe (SUPPORTS / REFUTES / NOT ENOUGH INFO)
ECE (Expected Calibration Error)	0,1570	Écart entre la confiance annoncée et la fiabilité réelle du modèle NLI

Exemples d'erreurs observées

L'analyse des affirmations mal classées montre plusieurs cas où le modèle NLI est très confiant (proche de 100 %) sur un label pourtant incorrect, révélant un problème de sur-confiance plutôt qu'un simple manque de preuves :

Affirmation	Label réel	Label prédit	Confiance
L.A. Reid has served as the president of a record label.	SUPPORTS	REFUTES	99,99 %
In 1930, Albert S. Ruddy is born.	SUPPORTS	REFUTES	100,00 %
Alexandra Daddario was born in 1986.	SUPPORTS	REFUTES	100,00 %

10. Difficultés rencontrées

- Volume important des données Wikipedia à traiter, avec un risque de saturation mémoire lors du prétraitement.
- Génération des embeddings sur un grand nombre de passages, nécessitant une gestion efficace des lots et du stockage.
- Gestion de l'environnement et des dépendances d'apprentissage automatique (PyTorch, Transformers, Sentence-Transformers, FAISS) : leur installation directe sur l'environnement Windows du poste de développement s'est révélée impossible, ce qui a nécessité l'utilisation d'un environnement conda dédié sous WSL (Windows Subsystem for Linux).
- Temps de calcul important en l'absence de GPU : le chargement des modèles (environ 67 secondes) et le traitement d'une affirmation (20 à 80 secondes observées) restent lents pour un usage interactif.

11. Solutions apportées

- Lecture en streaming des fichiers Wikipedia et génération progressive des chunks (article par article), afin de limiter l'empreinte mémoire du prétraitement.
- Encodage des embeddings par lots (BATCH_SIZE = 512) et sauvegarde fragmentée en plusieurs fichiers .npy accompagnés d'un fichier de métadonnées, plutôt qu'un unique fichier volumineux.
- Mise en place d'un environnement conda dédié (« fact-checker ») sous WSL, isolant les dépendances d'apprentissage automatique de l'environnement Windows natif.
- Ajout de nouvelles métriques d'évaluation (Macro F1 et Evidence Precision) dans evaluation/metrics.py et leur intégration dans evaluation/evaluate_pipeline.py, afin de compléter l'analyse de performance du système.
- Utilisation d'un index FAISS de type IndexFlatIP combiné à des embeddings normalisés, permettant une recherche par similarité cosinus efficace pour l'étape de recherche, indépendamment du coût plus élevé de l'étape de vérification NLI.

12. Limites

- Dépendance à Wikipedia : le système ne peut vérifier que des affirmations dont les preuves sont présentes dans le corpus indexé ; il ne consulte aucune source externe ni le Web en temps réel.
- Couverture limitée du corpus : dans la configuration actuelle du prétraitement (MAX_FILES = 2 dans indexing/preprocess_wikipedia.py), seul un sous-ensemble des fichiers Wikipedia est traité, pour un total de 493 500 documents distincts indexés lors de nos tests. De nombreuses affirmations du jeu FEVER ont dû être ignorées lors de l'évaluation car leurs preuves de référence ne figuraient pas dans le corpus indexé.

- Performances variables selon les affirmations : sur l'échantillon évalué, le système atteint 64 % d'exactitude et 38 % de FEVER Score, avec des cas de sur-confiance du modèle NLI sur des prédictions erronées.
- Faible précision des preuves (Evidence Precision de 3,69 % sur l'échantillon testé) : parmi les passages retenus après reranking, seule une faible proportion correspond exactement à une phrase de preuve de référence.
- Temps de calcul : en l'absence de GPU, le traitement d'une affirmation prend de 20 à 80 secondes environ (hors chargement des modèles), ce qui limite l'usage en temps réel à grande échelle.
- Langue unique : le corpus documentaire et les modèles utilisés ne traitent que l'anglais, comme indiqué explicitement dans l'interface utilisateur.

13. Perspectives

- Support de plusieurs langues, au-delà de l'anglais actuellement requis.
- Recherche Web en temps réel, en complément du corpus Wikipedia statique actuellement indexé.
- Utilisation de modèles de langage plus récents, notamment pour améliorer la robustesse du module de vérification NLI et réduire les cas de sur-confiance observés.
- Amélioration du pipeline RAG (recherche augmentée), par exemple en enrichissant la stratégie de reranking ou en indexant l'intégralité du corpus Wikipedia plutôt qu'un sous-ensemble.
- Déploiement dans le cloud, pour un accès mutualisé et une meilleure disponibilité.
- Exposition du pipeline via une API (par exemple avec FastAPI), pour permettre une intégration dans d'autres applications au-delà de l'interface Streamlit actuelle.

Conclusion

Ce projet a permis de concevoir et de mettre en œuvre un système complet de vérification automatique de faits, depuis le prétraitement du corpus Wikipedia jusqu'à la restitution d'un verdict argumenté via une interface utilisateur. L'ensemble des objectifs spécifiques fixés a été atteint : prétraitement du corpus, construction de la base documentaire et de l'index FAISS, recherche et tri des preuves pertinentes, vérification par un modèle NLI, et présentation des résultats sous forme de verdict, de preuves et de score de confiance.

L'évaluation menée sur un échantillon du jeu de données FEVER (50 affirmations) montre des résultats encourageants pour un prototype (64 % d'exactitude, 56 % de rappel des preuves, 38 % de FEVER Score), tout en révélant des marges de progression importantes, notamment sur la précision des preuves retrouvées (3,69 %) et sur la confiance parfois mal calibrée du module de vérification (ECE de 0,157).

Les limites identifiées — couverture partielle du corpus, dépendance à Wikipedia, temps de calcul sur CPU — ouvrent des perspectives claires d'amélioration : élargissement du corpus indexé, intégration de sources complémentaires, adoption de modèles plus récents, et déploiement à plus grande échelle. Ce travail constitue ainsi une base fonctionnelle pour un système de fact-checking assisté par IA, tout en illustrant concrètement les défis techniques propres aux architectures de type recherche augmentée (RAG).